



Systemes Numériques / FPGA

SÉBASTIEN VAN
CAUWENBERGHE

(S3V@ECAM.BE)

Sommaire



Lesson 1. Introduction aux FPGA et logique combo



Lesson 2. Logique sequentielle



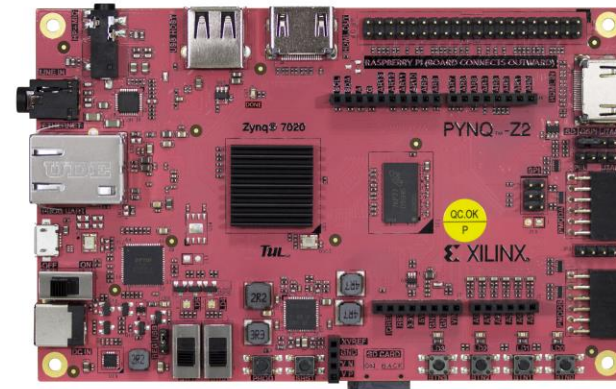
Lesson 3. Interfaces



Lesson 4. Pipelines



Lesson 5. Fun stuff



Introduction

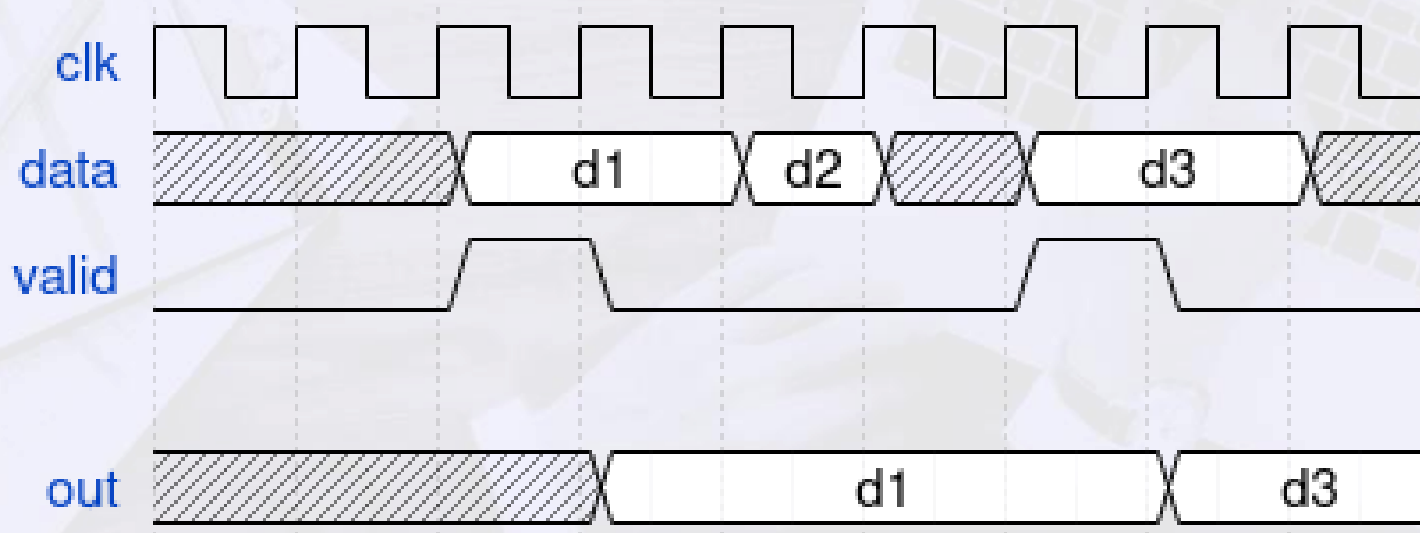
FPGA interfaces:

- Primitives de base
- Interfaces Internes
- Interfaces externes
- Datapath



Primitives utilisées dans les bus

- Blocs de base pour la fabrication de bus internes



- Le tout est de savoir quand la donnée est correcte, toutes les transitions ne signifient pas des données valides

Primitives utilisées dans les bus - Stream

- Base de certains bus, base de certains modes de transfert de données

- Données + Valid (+ Ready optionnel)

- Valid permet de savoir quand une donnée est valide

- Ready permet de savoir quand le bloc peut accepter une nouvelle donnée

- Règle: Quand Valid est mis à 1, il ne peut être remis à 0 qu'après un Ready.

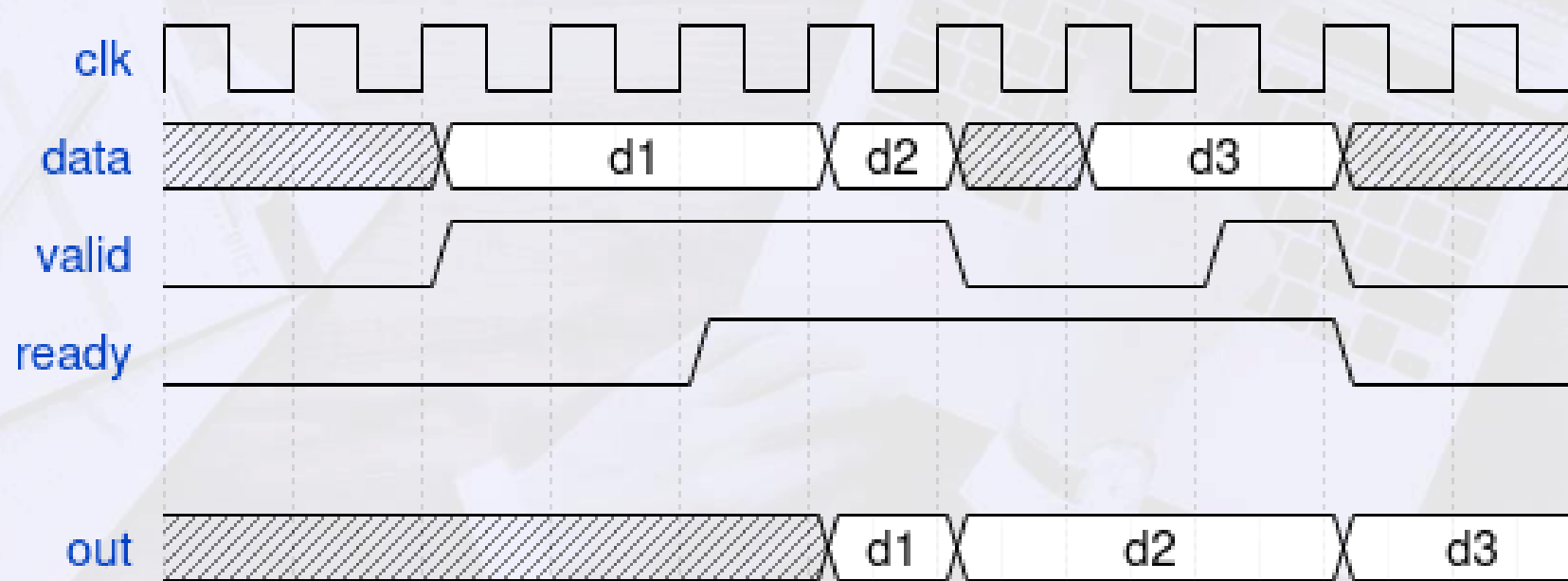
- Ready peut être mis à 1 n'importe quand

- Hypothèse: Les données sont valides et acceptées quand

Valid et Ready sont à 1

- Application : AXI4, AXI4-lite

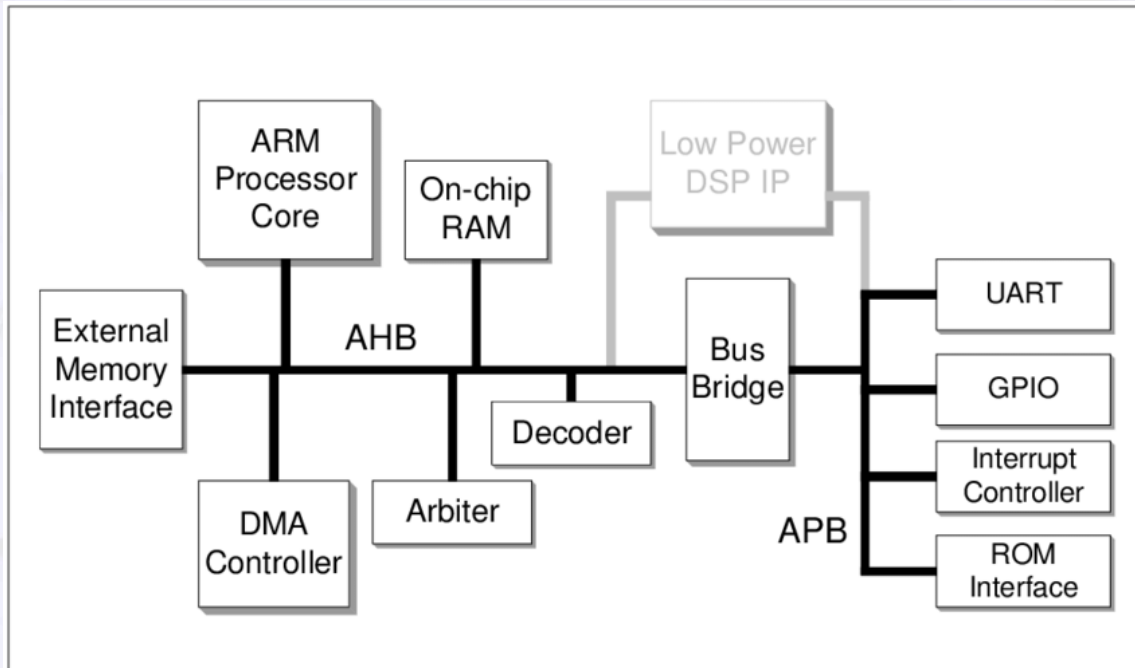
Primitives utilisées dans les bus - Stream



Primitives utilisées dans les bus - Stream Applications

- Traitement de données, images, audio
- Bus Memory-mapped (AXI4)
- Interface native des FIFO
- Traitement par stream ou par packet

Memory Mapped busses



- Classic SoC architecture, example with ARM CPU
- AXI4/AHB for high-speed, high-throughput interconnect
- APB for simple peripherals control/status
- Address translation handled by Bus Bridge

Memory mapped - APB

2.1 AMBA APB signals

Table 2-1 lists the APB signals.

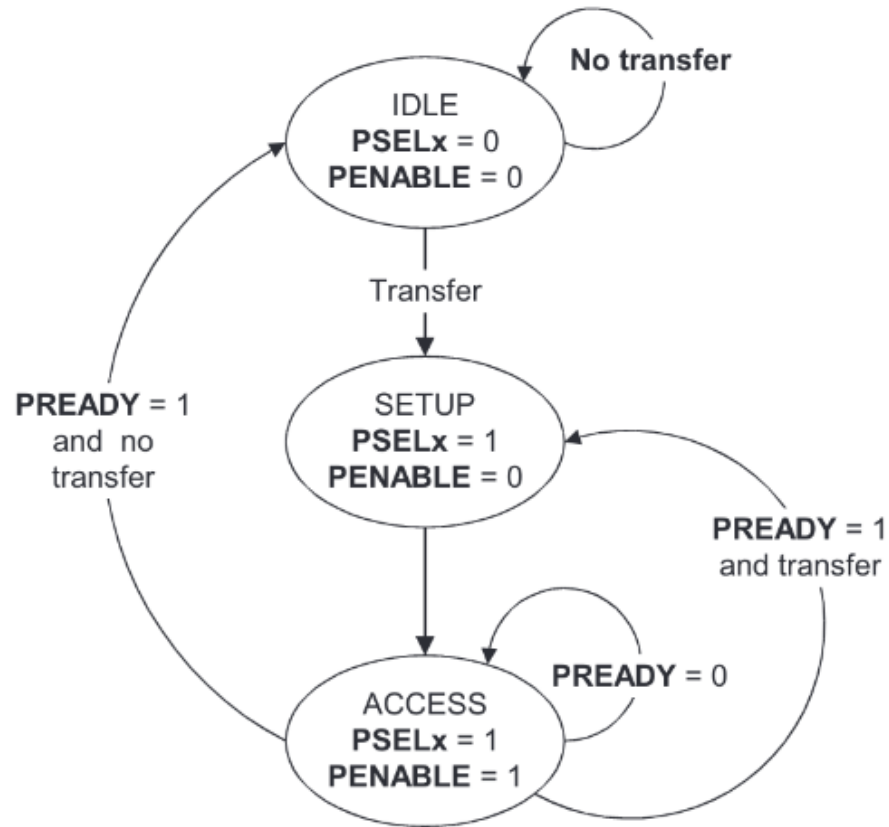
Table 2-1 APB signal descriptions

Signal	Source	Description
PCLK	Clock source	Clock. The rising edge of PCLK times all transfers on the APB.
PRESETn	System bus equivalent	Reset. The APB reset signal is active LOW. This signal is normally connected directly to the system bus reset signal.
PADDR	APB bridge	Address. This is the APB address bus. It can be up to 32 bits wide and is driven by the peripheral bus bridge unit.
PPROT	APB bridge	Protection type. This signal indicates the normal, privileged, or secure protection level of the transaction and whether the transaction is a data access or an instruction access.
PSELx	APB bridge	Select. The APB bridge unit generates this signal to each peripheral bus slave. It indicates that the slave device is selected and that a data transfer is required. There is a PSELx signal for each slave.
PENABLE	APB bridge	Enable. This signal indicates the second and subsequent cycles of an APB transfer.
PWRITE	APB bridge	Direction. This signal indicates an APB write access when HIGH and an APB read access when LOW.
PWDATA	APB bridge	Write data. This bus is driven by the peripheral bus bridge unit during write cycles when PWRITE is HIGH. This bus can be up to 32 bits wide.
PSTRB	APB bridge	Write strobes. This signal indicates which byte lanes to update during a write transfer. There is one write strobe for each eight bits of the write data bus. Therefore, PSTRB[n] corresponds to PWDATA[(8n + 7):(8n)] . Write strobes must not be active during a read transfer.
PREADY	Slave interface	Ready. The slave uses this signal to extend an APB transfer.
PRDATA	Slave interface	Read Data. The selected slave drives this bus during read cycles when PWRITE is LOW. This bus can be up to 32-bits wide.
PSLVERR	Slave interface	This signal indicates a transfer failure. APB peripherals are not required to support the PSLVERR pin. This is true for both existing and new APB peripheral designs. Where a peripheral does not include this pin then the appropriate input to the APB bridge is tied LOW.

Reference :

<https://developer.arm.com/documentation/ih0024/c>

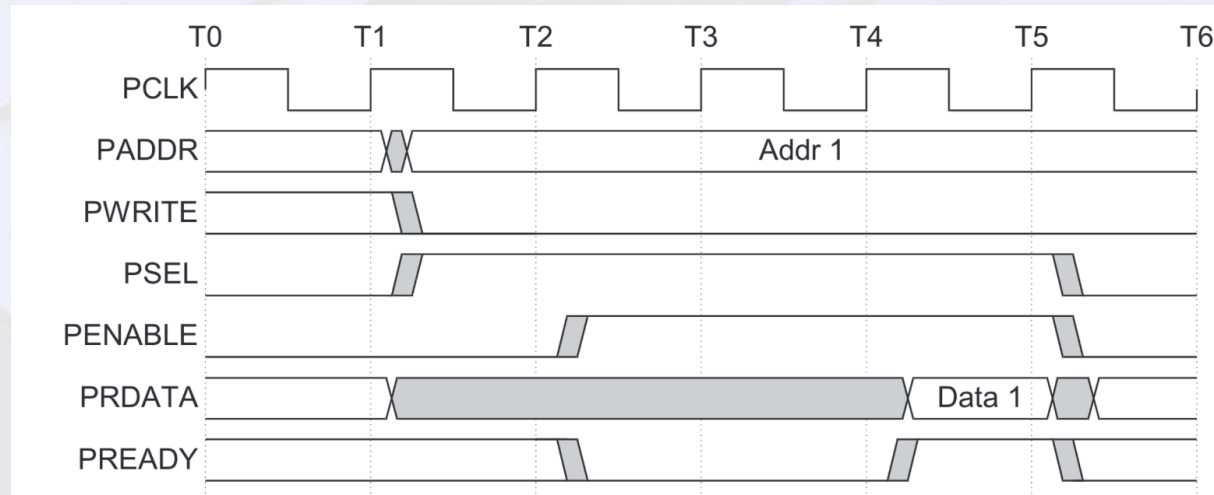
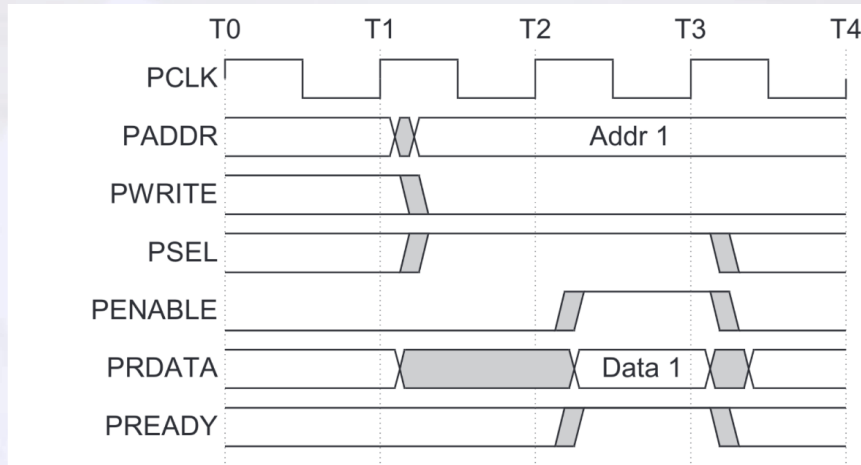
Memory mapped - APB



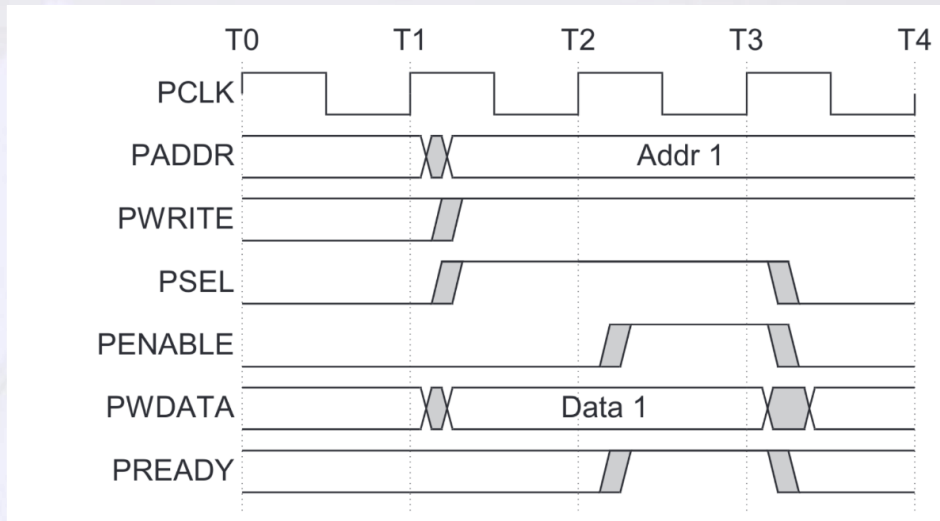
- IDLE: Nothing going on waiting for transfer
- SETUP: prepare address
- ACCESS: operate on data
- Possible to do Back-to-back operations
- This FSM describes the Master POV
- Slave implementation DOES not require state machine and it's more robust this way.

Memory mapped – APB Read

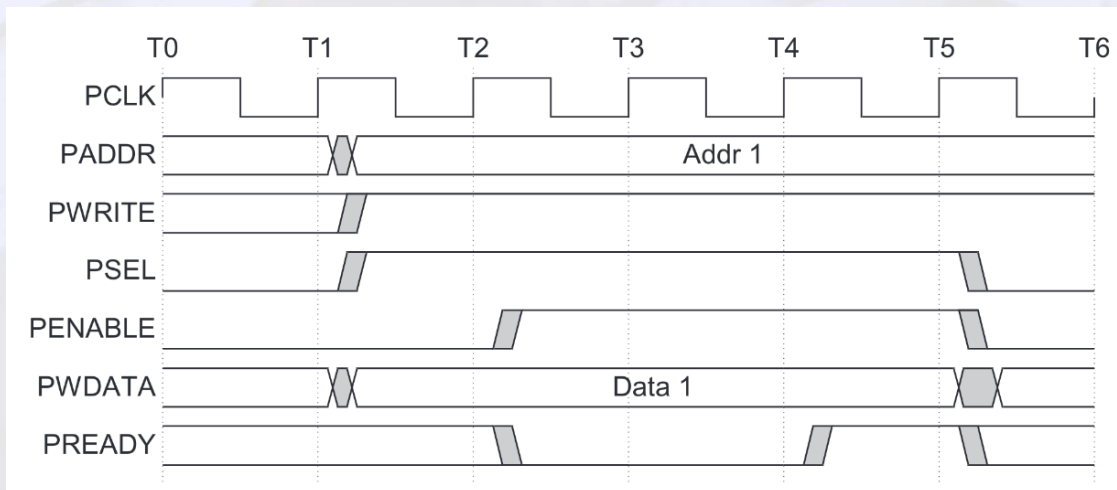
- See how APB can be seen as Stream
- Maps naturally to RAM accesses/registers



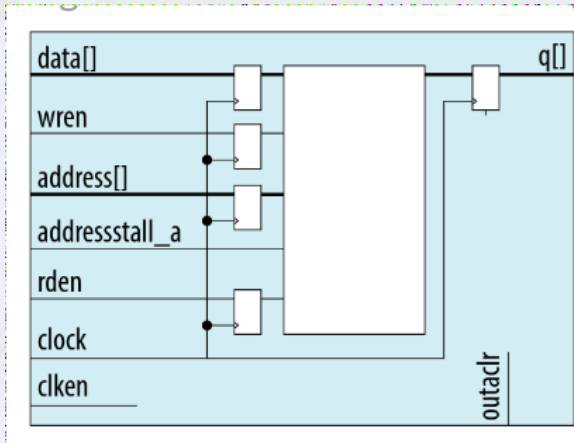
Memory mapped – APB write



- Address and Data are presented on the same cycle for a write
- The SETUP state here is decoration

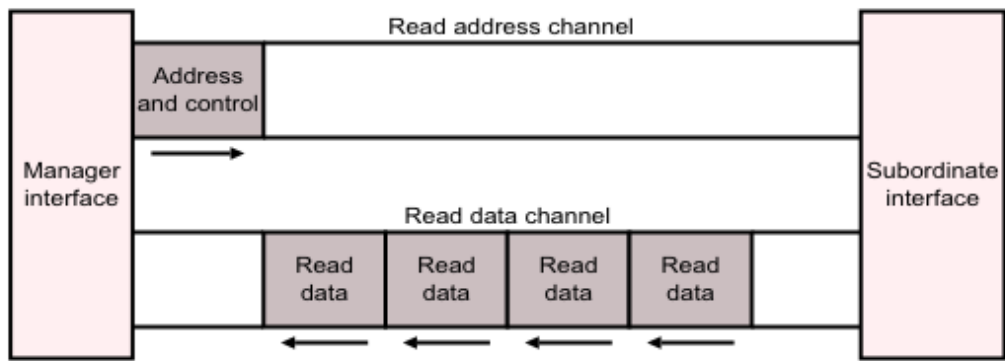
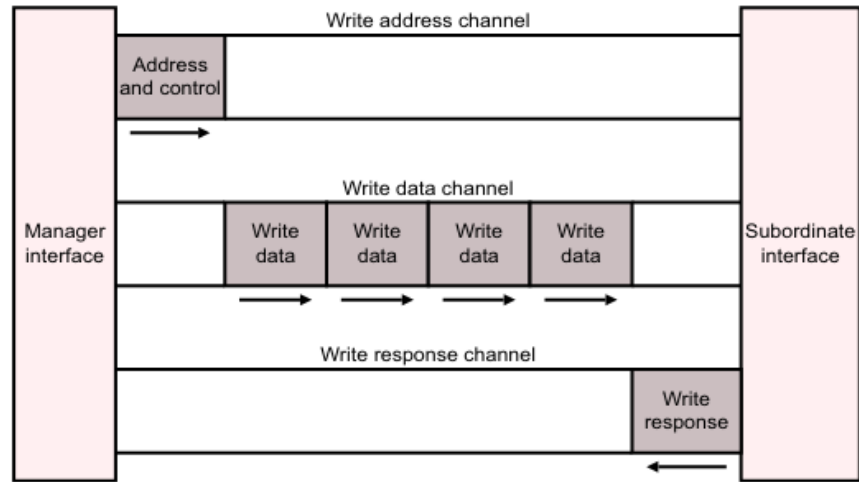


Memory mapped – FPGA memory Intel example



- Block RAM example from Intel MAX10 M9k block
- 1 set of registers at the input
- Output register is optional
- Data[] is input, Q[] is output
- Source
: <https://www.intel.com/content/www/us/en/docs/programmable/683431/current/ram-1-port-ip-core-references.html>

Memory mapped – AXI5-lite



- Has 5 streams (!!!)
- Higher performance than APB
- Much more complex
- Comes mainly pre-made for IPs
- Source
: <https://developer.arm.com/documentation/ih0022/latest>

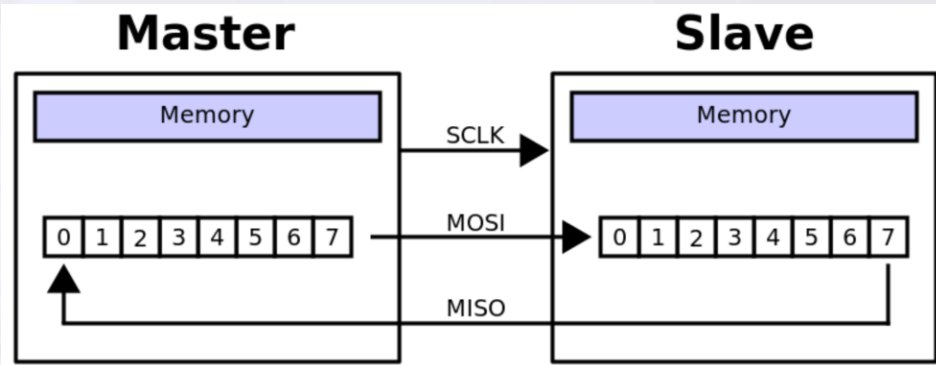
Table 0-20 AXI5-Lite signals

Global	Write address channel	Write data channel	Write response channel	Read address channel	Read data channel
ACLK	AWVALID	WVALID	BVALID	ARVALID	RVALID
ARESETn	AWREADY	WREADY	BREADY	ARREADY	RREADY
AWAKEUP	AWADDR	WDATA	-	ARADDR	RDATA
-	AWPROT ^a	WSTRB	BRESP	ARPROT ^a	RRESP
-	AWID	-	BID	ARID	RID
-	AWIDUNQ	-	BIDUNQ	ARIDUNQ	RIDUNQ
-	AWSIZE	-	-	ARSIZE	-
-	AWUSER	WUSER	BUSER	ARUSER	RUSER
-	AWTRACE	WTRACE	BTRACE	ARTRACE	RTRACE
-	-	WPOISON	-	-	RPOISON

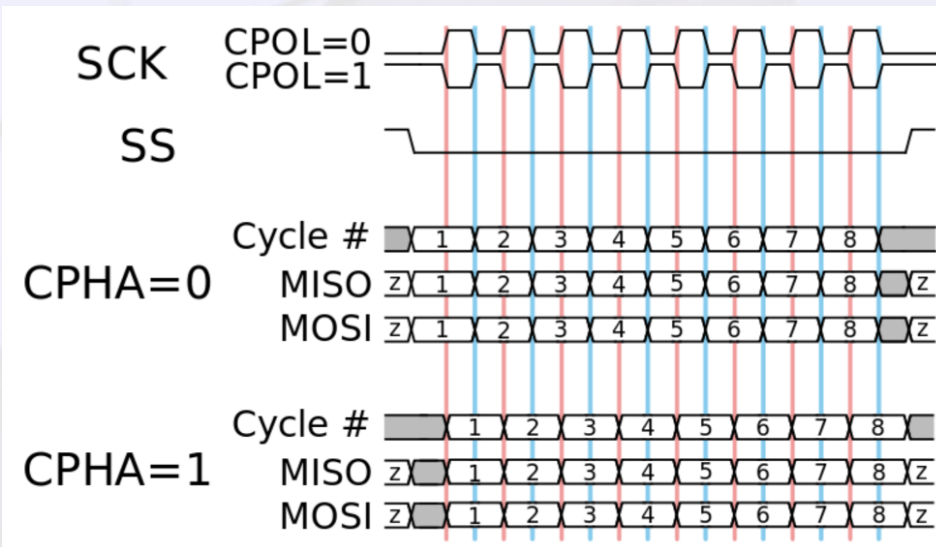
External interfaces

- Asynchronous Serial Interface (UART, ...)
- Synchronous Parallel interfaces (Memory, ADC, Ethernet, ...) SDR or DDR
- Synchronous Serial interfaces (I2S, I2C, SPI, ...)
- Hi-speed serial interfaces (PCIe, USB, HDMI, ...)
- Fake analog (Use digital pins for Analog IOs)
- Use IPs designed by someone else (even opensource)
- Do not reinvent the wheel

External interfaces - SPI



- Parallel to serial conversion on MOSI/MISO
- Possibly Multi-bit, Dual/Quad/Octal SPI
- Depending of performance requirements, implementation will change
- Slow speed version (some MHz) : Master will use FSM to sequence operations on multiple cycles. Slave will double flop inputs and detect edges.
- High speed (50MHz+) : Do all in one cycle and let the synthesizer handle timings. Touchy subject :).



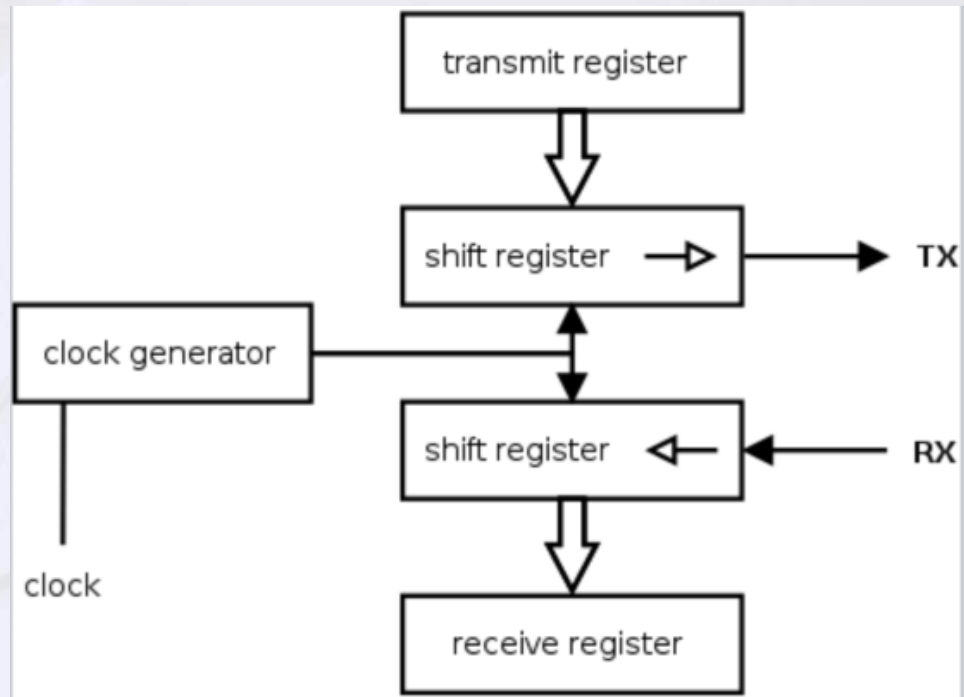
LTC2452

TIMING CHARACTERISTICS

The ● denotes the specifications which apply over the full operating temperature range, otherwise specifications are at $T_A = 25^\circ\text{C}$.

SYMBOL	PARAMETER	CONDITIONS	MIN	TYP	MAX	UNITS
t_{CONV}	Conversion Time		● 13	16.6	23	ms
f_{SCK}	SCK Frequency Range		●		2	MHz
t_{LSCK}	SCK Low Period		● 250			ns
t_{HSCK}	SCK High Period		● 250			ns
t_1	$\overline{\text{CS}}$ Falling Edge to SDO Low-Z	(Notes 7, 8)	● 0		100	ns
t_2	$\overline{\text{CS}}$ Rising Edge to SDO Hi-Z	(Notes 7, 8)	● 0		100	ns
t_3	$\overline{\text{CS}}$ Falling Edge to SCK Falling Edge		● 100			ns
t_{K0}	SCK Falling Edge to SDO Valid	(Note 7)	● 0		100	ns

External interfaces - UART



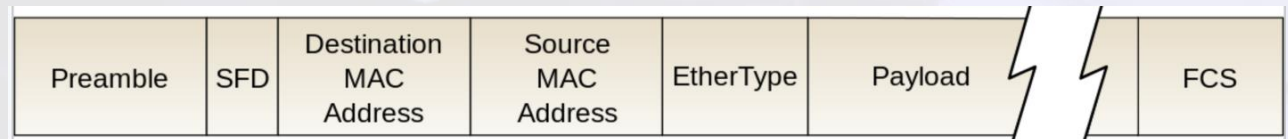
- Clock generator at 2x baudrate
- FSM to control the shift registers
- Synchronize RX acquisition with the start bit falling edge

External interfaces - Ethernet

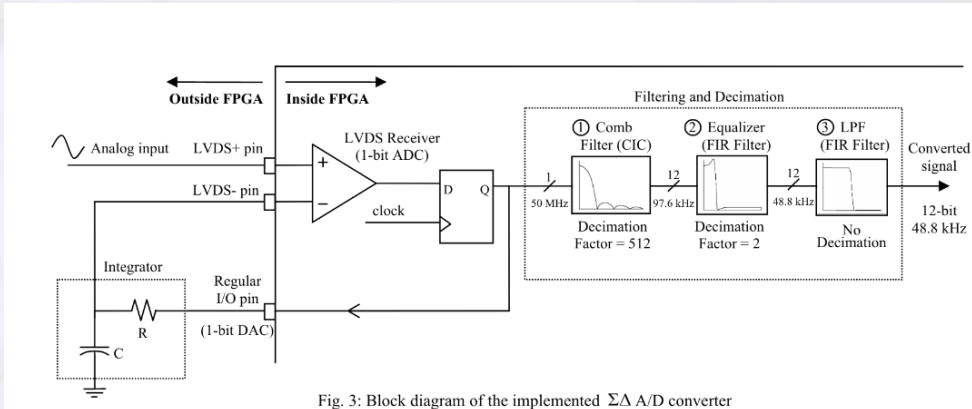
Signal name	Description
GTXCLK	Clock signal for gigabit TX signals (125 MHz)
TXCLK	Clock signal for 10/100 Mbit/s signals
TXD[7..0]	Data to be transmitted
TXEN	Transmitter enable
TXER	Transmitter error (used to intentionally corrupt a packet, if necessary)

Signal name	Description
RXCLK	Received clock signal (recovered from incoming received data)
RXD[7..0]	Received data
RXDV	Signifies data received is valid
RXER	Signifies data received has errors
COL	Collision detect (half-duplex connections only)
CS	Carrier sense (half-duplex connections only)

- Ethernet MAC is in the FPGA
- Ethernet PHY is in a discrete chip
- TXCLK/RXCLK allow the sequencing of the RX/TX data from the (R)(G)MII interface.
- Data is transferred with TXD/RXD conditioned by TXEN, RXDV
- You can use FPGA to send your own packets following Ethernet standard for example
- Source : https://en.wikipedia.org/wiki/Media-independent_interface



External interfaces – Analog Hacks



- Abuse all digital interfaces (LVDS)
- Average resolution
- Good for FPGA frying
- Prefer use of real ADCs
- Source: http://www.ee.nmt.edu/~erives/531_14/Sigma-Delta.pdf